Programming Project 2 – **RPC-Based Multi-Threaded Server** [70 pts]

COMP7212 Operating/Distributed Systems

Due: 11:59PM on 10/31/2026

1. Project Objective

In this project, students will extend the Multi-Threaded Echo Server developed in Programming Project 1-2 to implement a simple Remote Procedure Call (RPC) system. Students will learn how a client can invoke procedures on a remote server through network communication, including parameter marshaling and unmarshaling, request processing, and response transmission.

2. Project Description

In Programming Project 1-2, you implemented a multi-threaded TCP server that receives messages from multiple clients and echoes them back. In this project, you will extend your previous implementation to support remote procedure calls. Instead of simply echoing received messages, the server will execute a procedure requested by a client and return the result. Your implementation must satisfy the following requirements.

2.1. Multi-Threaded Server

Reuse the multi-threaded server architecture from Project 1-2.

- The server must continuously listen for incoming TCP connections.
- The server must create a separate thread for each connected client.
- The server must support multiple clients simultaneously.
- Each client must be able to make multiple RPC requests over its connection.

2.2. Remote Procedure Calls

The server must support the following remote procedures:

1. ADD(a, b): Return the sum of two integers.
2. SUBTRACT(a, b): Return the difference between two integers.
3. MULTIPLY(a, b): Return the product of two integers.
4. MAX(a, b): Return the larger of two integers.

For example, a client may request ADD(10, 20), and the server should execute the requested procedure and return 30. ***All computations must be performed on the server, not on the client.***

2.3. Parameter Marshaling and Unmarshaling

Implement a simple communication protocol that specifies how RPC requests and responses are formatted. For example, an RPC request may use the following format:
ADD,10,20
The client must marshal the procedure name and its parameters into a message before sending it to the server. The server must unmarshal the received message to identify the requested

procedure and its parameters. After executing the procedure, the server must marshal the result into a response message and send it back to the client. The client must unmarshal the response and display the returned result. You must define a consistent message format and ensure that the client and server correctly handle individual messages transmitted over TCP.

2.4. Client-Side RPC Interface
Implement a client-side interface that allows the user to invoke remote procedures.
For example:
Enter operation: ADD
Enter first integer: 10
Enter second integer: 20
Result from server: 30

The client should internally perform the following operations:

1. Marshal the procedure name and parameters.
2. Send the RPC request to the server.
3. Wait for the server's response.
4. Unmarshal the response.
5. Display the result.

The client must not perform the requested calculation locally.

2.5. Error Handling
The server must properly handle invalid RPC requests.
For example:
DIVIDE,10,2
Since DIVIDE is not a supported procedure, the server should return an error message instead of terminating. The server must also handle invalid parameters and unexpected client disconnections without affecting other connected clients.

3. Testing Your RPC Server

You must demonstrate the following scenarios:

1. Single Client: Execute all four remote procedures and verify the results.
2. Multiple Requests: Send several RPC requests from the same client without reconnecting.
3. Invalid Procedure: Request a procedure that does not exist and verify that the server returns an appropriate error.
4. Client Disconnection: Disconnect one client and verify that the server continues serving other clients.

4. What to Submit

- A document explaining how to compile and execute your project.
- Source code for the RPC server and client.

- Screenshots showing successful execution of the required test scenarios.
- A short explanation of how parameter marshaling and unmarshaling are implemented.